

The Chocolate Bar Problem

→ m chocolate bars of varying integer lengths.

= array of integers with length m : start, s

→ n hungry children who want differing integer amounts of chocolate
↳ distinct?

= array of integers with length n : target, t

Aim: feed all children the correct amount while taking fewest cuts to chocolate

= get t from s by repeatedly partitioning each integer element s_i into two new elements $s_{i,a}$ and $s_{i,b}$, where $s_i = s_{i,a} + s_{i,b}$.

→ Fewest cuts implies shortest path = BFS!

Initial Thoughts...
→ every new piece is an integer
→ each cut may happen at any integer position on any piece.
→ one piece cut per operation

→ $\sum t_i \leq \sum s_i$ must be true.

→ ~~$m \leq n$~~ NOT TRUE! e.g. $s = \{3, 4\}$, $t = \{3\}$ → edge case of no cuts!

Provided example: $s = \{2, 5, 7\}$; $t = \{4, 3, 2, 1\}$

Minimum cuts; $n_{\min} = 2$

Depth = 0: $s^0 = \{2, 5, 7\}$

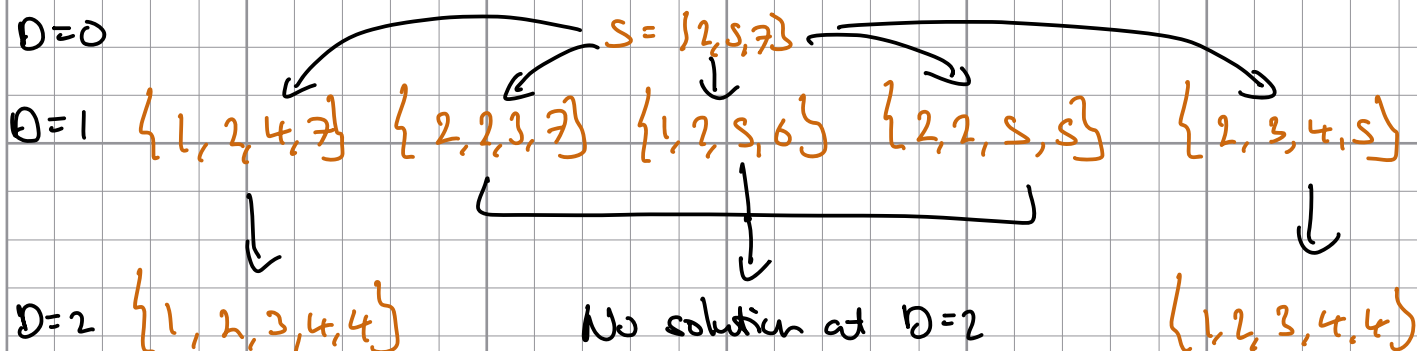
Depth = 1, $k = 2$: $s'_0 = \{2, 1, 4, 7\}$, $s'_1 = \{5, 2, 5, 7\}$, $s'_2 = \{2, 3, 2, 7\}$
 $s'_3 = \{2, 4, 1, 7\}$, $s'_4 = \{2, 5, 1, 6\}$, $s'_5 = \{2, 5, 2, 5\}$, ...

Note straight away that if ordering is irrelevant, then

there are repeated nodes, e.g.: $S'_1 = S'_2$ → sort to ensure a single representation of each!

And, of course, you can get to t at Depth=2 from any node of S' that results in t , e.g.

$$S'_0 = \{2, 1, 4, 7\} \rightarrow \{2, 1, 4, 3, 4\} = \{6, 4\}$$



Assumptions to check with Jake:

- 1) Cuts means $a = b + c$, where $a, b, c \geq 1$ are integers?
- 2) Does every child's piece have to come from one bar only?
- 3) Solution guaranteed? Deal with no solution?
- 4) Based on example, it appears to be okay if $\sum_j b_j < \sum_i s_i$, i.e. there is no requirement that $\sum_j b_j = \sum_i s_i$? and a remainder is allowed?
- 5) Does ordering of elements matter?
- 6) Are duplicates allowed in t ? i.e. must each element of t be distinct?

Questions before starting:

- Does a reverse search (summing the elements of t + remainder to arrive at s) provide any benefit?
- Can I optimize any BFS with Threading?

There is a problem here.

We need to know the remainder in advance which we can get from $r = \sum s_i - \sum t_j$. However, we don't know the distribution of r . Consider: $S = \{6, 6\}$, $t = \{4, 4\} \Rightarrow \underline{r = 4}$. But, t is obtained by $S \Rightarrow \{4, 2, 4, 2\} = \{t, 2, 2\}$

$\Rightarrow r = 2+2=4$, but we don't know this in advance.

Would have to cycle through remainder partitions.

Coming back to this idea

$\Rightarrow$ exact solution has exponential complexity!

Warned in question

$\hookrightarrow$ Explore approximations without exact solution / remainder.

$\hookrightarrow$ Start from target and work backwards?

$\hookrightarrow$ Match targets to bars.

$\hookrightarrow$ Find targets that can fit inside bars

e.g.

Start = $[2, 5, 7]$, target = $[4, 3, 2, 1]$

assigned possibilities = $[5, 7]$

requires target sorted largest to smallest

→ Take minimum difference as approximation?

e.g. 4 → min diff = 5, remain = [2, 1, 7], cuts = 1
3 → min diff = 7, remain = [2, 1, 4], cuts = 2
2 → min diff = 2, remain = [0, 1, 4], cuts = 2
1 → min diff = 1, remain = [0, 0, 4], cuts = 2

⇒ Where does this approximation break down?

→ start = [3, 4], target = [2, 2]

Optimal solution is clearly 1 cut 4 → [2, 2].

But, following min diff approximation:

2 → min diff = 3, remain = [1, 4], cuts = 1
2 → min diff = 4, remain = [1, 2], cuts = 2

⇒ Min-diff approach will likely be much better time complexity for approximate solution:

- Requires target sorted largest to smallest → $O(n \log n)$
- Will require scan through start (m) for each element in target to find possibilities: $O(mn)$

→ Total = $O(mn + n \log n)$

13/08/26 - Negley from Tale to questions on assumptions combinatorics.

Biggest concern - wrong assumption in 2). Child's chocolate does not have to come from one bar.
↳ Need new algorithm!

Secondary concern - treating no solution. Can we approximately feed all children instead of none?

⇒ More flexibility with children fed from different bars

↳ Only one no solution scenario ⇒ sum(start) < sum(target)
↳ ~~The following must be true...~~

Let $d_{ij} \geq 0$ be the integer amount from chocolate bar i to child j :

⇒ $\sum_j d_{ij} \leq s_i$ → cannot take more than bar contains

$\sum_i d_{ij} = t_j$ → each child gets their amount

Total amount distributed: $\sum_i \sum_j d_{ij} = \sum_j t_j$ Linear system

What about adding a dummy target so that $\sum_i s_i = \sum_j t_j$:

→ Let $\tilde{t}_j = \sum_i s_i - t_j$

Then $\tilde{t} = t + [\tilde{t}_j]$

e.g. $s = [2, 3, 7]$; $\tilde{t} = [4, 3, 2, 1]$ (4) → dummy child unused chocolate.

Let's go back to matching (remember matching targets to get values from solve 3).

If a bar s_i feeds p children:

→ No remainder of $s_i = \underline{p-1 \text{ cuts}}$

→ w/ remainder = p cuts

Best outcome in zero cuts, $p=1$ with no remainder

ie → $s_i = t_j$

What remainder does each target vs that result in?

(Add in dummy target as above)

⇒ Work through remainder from smallest to biggest.

	4	3	2	1	4
2	X	X	0	1	X
5	1	2	3	4	1
7	3	4	5	6	3

Matching order:

0 cuts $\rightarrow$ n starts, 1 target (cap n)
 1 cut $\rightarrow$

	4	3	2	1	4
2	X	X	0	1	X
5	1	2	3	4	1
7	3	4	5	6	3

Order to follow:

- 1) 0 cuts $\rightarrow$ 1 $\leftrightarrow$ 1
- 2) 0 cuts $\rightarrow$ 2 $\leftrightarrow$ 1
- 3) 1 cut $\rightarrow$ 1 $\leftrightarrow$ 2
- 4) Approximate $\rightarrow$ log R. $\rightarrow$ find 0 cuts.

	3	3	6
1	X	X	X
2	4	4	1
7	4	4	1
8	5	5	2

7 $\rightarrow$ 4
 4
 8

7 $\rightarrow$ 1 3 4 7 2 6

3 3 4 7 6